

Patient Intake Assistant

A five-node LangGraph state machine, wrapped in FastAPI and React, built on top of PatientAgentBench's real healthcare sandbox rather than a reimplementaion of it.

LangGraph · StateGraph

FastAPI + WebSocket

React + TypeScript

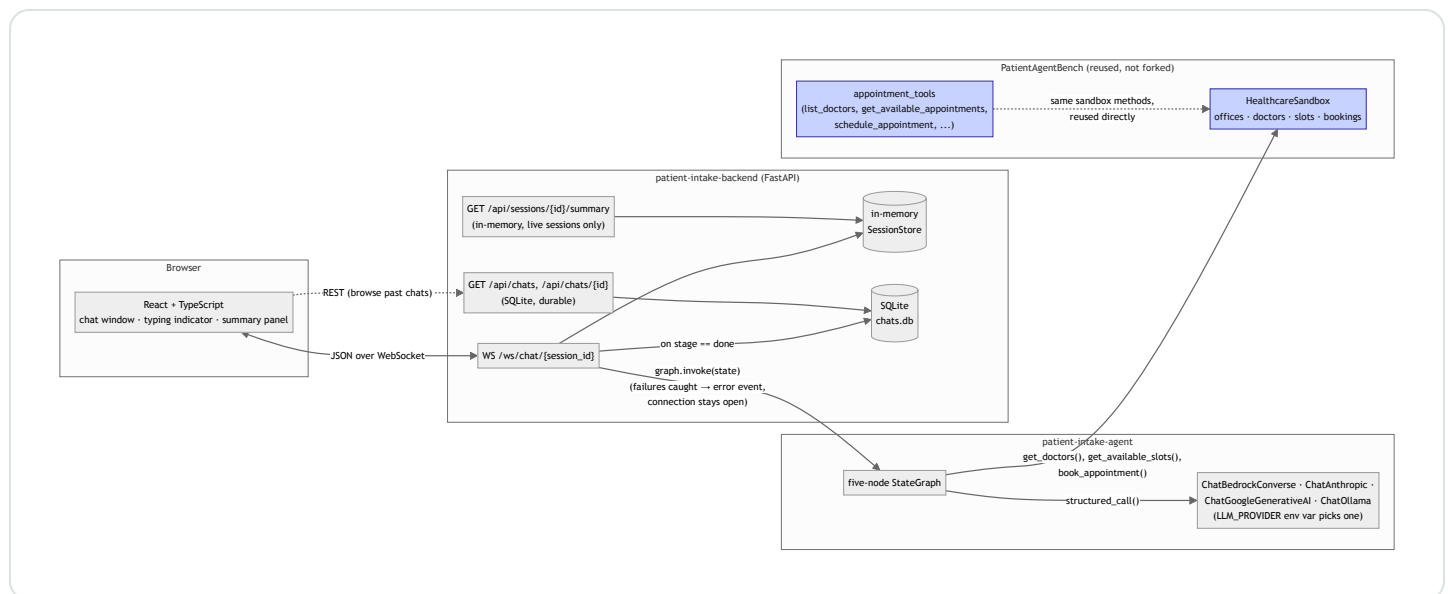
SQLite

Bedrock · Anthropic · Gemini · Ollama

PatientAgentBench (reused)

01 System overview

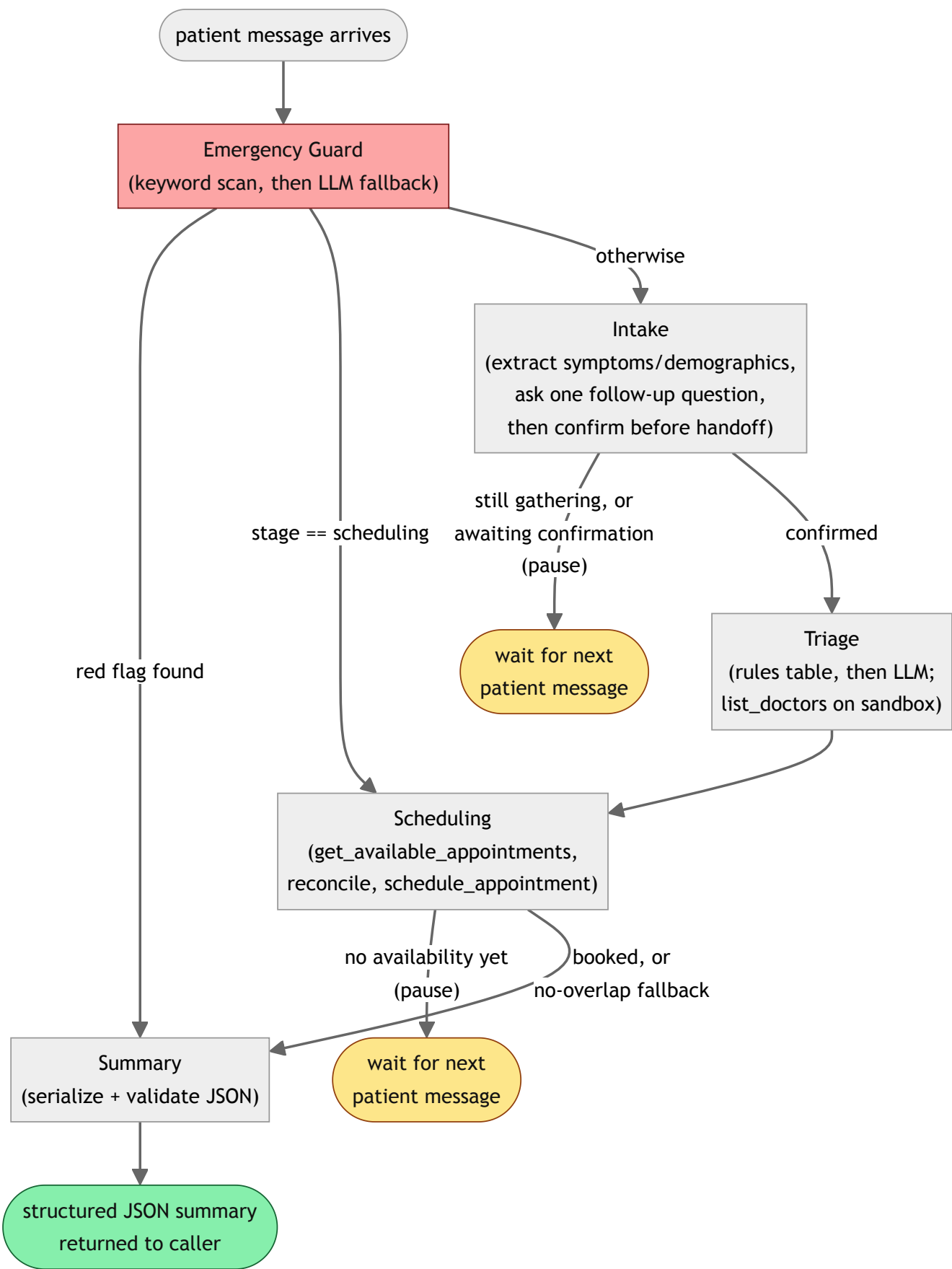
Four layers: a browser client, a FastAPI backend holding two deliberately separate stores, the LangGraph agent itself, and the slice of PatientAgentBench it reuses directly rather than forks.



The two blue boxes are **literally imported**, not reimplemented — `sandbox_setup.py` builds a real `HealthcareSandbox`, and Triage/Scheduling call its methods directly, the same methods PatientAgentBench's own tools call internally. The two data stores are intentionally separate: the in-memory store is fast but gone on restart; SQLite is written exactly once per session, when it reaches done, and survives one.

02 The five-node graph

Compiled once, invoked fresh per patient message — no cycles, no checkpoint. Emergency Guard is the actual entry point, ahead of Intake, so an obvious red flag is caught before any extraction call is spent on it.



Only two nodes ever pause — Intake and Scheduling. Triage always completes in the same turn as whatever handed it control. Emergency Guard sits on **every** path, including a direct route from mid-scheduling straight to Summary, which is what actually satisfies "runs on every patient turn, not just at the end."

03 Reused vs. built

What comes from PatientAgentBench unmodified, and what this project adds on top of it.

COMPONENT	ORIGIN	NOTES
reused HealthcareSandbox	PatientAgentBench	Offices, doctors, appointment slots, bookings — imported as a real dependency, not forked.
reused Appointment tools	PatientAgentBench	Triage/Scheduling call <code>get_doctors()</code> , <code>get_available_slots()</code> , <code>book_appointment()</code> directly.
built Emergency Guard	This project	No equivalent upstream — PatientAgentBench's only safety layer is a prompt instruction, no code-level check.
built Intake, Triage routing, Summary	This project	Upstream assumes an already-onboarded patient with a complete record; this project starts from an empty one.
built Five-node StateGraph	This project	Upstream's assistant is one flat ReAct loop; this is explicit multi-node routing with typed state.

04 Design notes

Stateless graph, stateful caller

No LangGraph checkpointer — deliberately mirrors PatientAgentBench's own pattern of resending accumulated state fresh each turn, rather than a persistent thread.

Cheapest check first

Emergency Guard and Triage both try a free, deterministic check (keyword scan, rules table) before ever calling an LLM.

Never let the LLM invent data

Triage clamps to a fixed specialty vocabulary; Scheduling only ever offers a real slot_id pulled from the live sandbox.

Failures are visible, not silent

A provider rate limit or timeout is caught per-turn and shown in the transcript — the connection and session state stay intact.